Announcements

Tentatively I'll be holding office hours Wednesday 12 - 2 (right before class).

- Working on getting space.

CC BY NC SA

A Look Back at Project 1

Before we get started, let's take a look back at Project 1's `removeFirst` method.

- We'll compare a student solution with my solution:

CC BY NC SA

A Look Back at Project 1

```
public T removeFirst() {
    if(!isEmpty()){
        T replace=sentinel.next.item;
       sentinel.next=sentinel.next.next;
       sentinel.next.prev=sentinel;
       size--;
       return replace;

    }
    return null;
}
```

```
public T removeFirst() {
  if (size == 0) {
    return null;
  }

  Node oldFront = sentinel.next;
  Node newFront = sentinel.next.next;

  sentinel.next = newFront;
  newFront.prev = sentinel;

  size -= 1;
  return oldFront.value;
}
```

What do you like better about mine (on the right)?

CC BY NC SA

Recommendations

Make your code obvious and easy to read.

- It's fine to create variables that serve only to give names to things (performance penalty is negligible or non-existent).

Use good style.

- Consistent spacing.
- Variable naming (camelCase in Java).
- Note: This will be mandatory starting a bit after the midterm.

```
public T removeFirst() {
  if (size == 0) {
    return null;
  }

  Node oldFront = sentinel.next;
  Node newFront = sentinel.next.next;

  sentinel.next = newFront;
  newFront.prev = sentinel;

  size -= 1;
  return oldFront.value;
}
```

CC BY NC SA

Announcements

Today's lecture is not in scope for midterm 1.

- But the ideas are useful, especially when we get to discussing TreeMaps, TreeSets, and Priority Queues later in the course.

CC BY NC SA

A Mini-Spoiler for Project 2

Similarly a lot of student code for project 2 looks like:

```
T removed = items[(nextLast - 1 + items.length) % items.length];
items[(nextLast - 1 + items.length) % items.length] = null;
```

Where the same non-obvious modulus math is repeated in many places. By contrast, my solution has:

```
private int wrapIndex(int index) {
   return (index + items.length) % items.length;
}
```

The code above would then be simplified as:

```
T removed = items[wrapIndex(nextLast - 1)];
items[wrapIndex(nextLast - 1)] = null;
```

CC BY NC SA



Lecture 11

Subtype Polymorphism, Comparators, Comparables

CS61B, Fall 2026 @ UC Berkeley

Josh Hug and Manuel Sabin

CC BY NC SA

Polymorphism vs. Function Passing

Lecture 11, CS61B, Fall 2026

CC BY NC SA

Polymorphism vs. Function Passing

Comparable and Comparators

- Comparables
- Compilation Error Puzzle
- Comparators
- Comparables vs. Comparators

Summary

Goals for Today

Today we're going to dig deeper into inheritance.

- We'll see the contrast between **polymorphism** and **function passing**.
- Unlike Python, where function passing is common, most idiomatic Java code relies more heavily on polymorphism.

Let's see some examples in Python.

CC BY NC SA

Polymorphism Example in Python

```
class Dog:
  def __init__(self, name, size):
    self.name = name
    self.size = size
  def __gt__(self, other):
    return self.size > other.size
```

```
hadi = Dog("hadi", 30)
zora = Dog("zora", 44)

# uses the __gt__ function
print(hadi > zora)
```

In the code above, the **Dog** class overloads the > operator.

- Operator overloading is a form of **polymorphism**.
- Polymorphism: “the ability in programming to present the same programming interface for differing underlying forms” [wiki]
- Idea: We can use the > operator to compare anything (i.e. “any underlying form”) in Python, including Dogs.
  - To define the behavior of >, we implement the function `__gt__(self, other)`

CC BY NC SA

Function Passing Example in Python

```
class Dog:
  def __init__(self, name, size):
    self.name = name
    self.size = size
  def __gt__(self, other):
    return self.size > other.size
```

```
def name_len(d):
  return len(d.name)

max(doglist, key=name_len)
```

In the code above, the **max** function accepts a key function.

- This is an example of **function passing**.

CC BY NC SA

Function Passing Example in Python

```
class Dog:
  def __init__(self, name, size):
    self.name = name
    self.size = size
  def __gt__(self, other):
    return self.size > other.size
```

```
def name_len(d):
  return len(d.name)

max(doglist, key=name_len)

# or using a lambda, do it on one line:
max(doglist, key=lambda d: len(d.name))
```

In the code above, the **max** function accepts a key function.

- This is an example of **function passing**.

CC BY NC SA

Goals for Today

```
class Dog:
  ...
  def __gt__(self, other):
    return self.size > other.size

hadi > zora
```

```
max(doglist, key=name_len)
```

Today: Polymorphism in Java

- Above, we see **polymorphism** on the left and **function passing** on the right.
- Unlike Python, where both are common, most idiomatic Java code relies more heavily on polymorphism.
  - Example: Want to specify how two things should be compared? Where Python uses key functions, Java (typically) uses inheritance!
  - Note: We won't even discuss function passing in Java in our course, but it does exist.

CC BY NC SA

Comparables

Lecture 11, CS61B, Fall 2026

Polymorphism vs. Function Passing

Comparable and Comparators

- **Comparables**
- Compilation Error Puzzle
- Comparators
- Comparables vs. Comparators

Summary

CC BY NC SA

Dogs

Consider the dogs below.

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new Dog("Grigometh", 200));
dogs.add(new Dog("Pelusa", 5));
dogs.add(new Dog("Clifford", 9000));
```



“Grigometh”, size: 200



“Pelusa”, size: 5



“Clifford”, size: 9000

CC BY NC SA

Dogs

To find the maximum, we can use `Collections.max`. Let's try this.

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new Dog("Grigometh", 200));
dogs.add(new Dog("Pelusa", 5));
dogs.add(new Dog("Clifford", 9000));
Dog maxDog = Collections.max(dogs);
```



"Grigometh", size: 200



"Pelusa", size: 5



"Clifford", size: 9000

CC BY NC SA

Dogs

To find the maximum, we can use `Collections.max`. Let's try this.

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new Dog("Grigometh", 200));
dogs.add(new Dog("Pelusa", 5));
dogs.add(new Dog("Clifford", 9000));
Dog maxDog = Collections.max(dogs);
```

This results in the incomprehensible error message below.

- The issue is that `max` doesn't know how to compare two `Dog` objects.

```
Required type:    Collection <? extends T>
Provided:         List       <Dog>

reason: no instance(s) of type variable(s) T exist so that Dog conforms to Comparable<? super T>
```

CC BY NC SA

Capability Specification in Java

Unlike Python, where we'd define a `__gt__` method to overload the > operator, in Java, we have to specify that a `Dog` is something that can be compared.

Question: How do we specify that a class has a certain capability in Java?

CC BY NC SA

Capability Specification in Java

Unlike Python, where we'd define a `__gt__` method to overload the > operator, in Java, we have to specify that a `Dog` is something that can be compared.

Question: How do we specify that a class has a certain capability in Java?

- We must implement some interface.

CC BY NC SA

The Comparable Interface

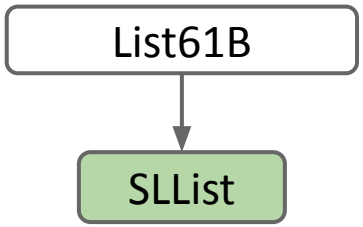
Unlike Python, where we'd define a `__gt__` method to overload the > operator, in Java, we have to specify that a `Dog` is something that can be compared.

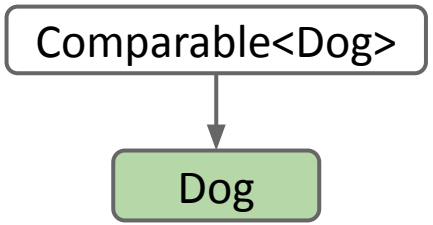
In Java, we will declare to the world that a `Dog` is-a `Comparable`.

- Just like how earlier we said an `SLList` is-a `List61B`.

Since almost all of Java is written in Java, we can go look at Comparable.java.

- Link on github: Comparable.java





CC BY NC SA

The Comparable Interface

Looking at Comparable.java, we see:

```
public interface Comparable<T> {
    /**
     * Compares this object with the specified object for order.
     * Returns a negative integer, zero, or a positive integer
     * as this object is less than, equal to, or greater than the
     * specified object.
     * ...
     */
    public int compareTo(T o);
}
```

Note: The full file is 142 lines long, almost all of which is documentation. This is not uncommon with really important parts of the library. Documentation is important.

CC BY NC SA

Implementing Comparable

Let's implement the `Comparable` interface in `Dog.java`. This means:

- Adding "`implements Comparable<Dog>`" to the top of the file.
- Overriding the `compareTo` method.

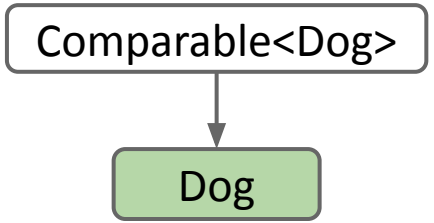
CC BY NC SA

Implementing Comparable

Let's implement the `Comparable` interface in `Dog.java`. This means:

- Adding "`implements Comparable<Dog>`" to the top of the file.
- Overriding the `compareTo` method.

```
public class Dog implements Comparable<Dog> {
    ...
    @Override
    public int compareTo(Dog uddaDog) {
        if (size > uddaDog.size) {
            return 1;
        }
        if (size < uddaDog.size) {
            return -1;
        }
        return 0;
    }
}
```



"uddaDog" courtesy of Harry Alu of Ehu & Kai Adventures (on the Big Island).

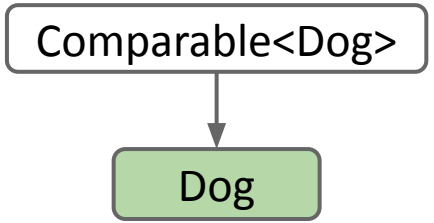
CC BY NC SA

Implementing Comparable (Better Approach)

A cleaner implementation is shown below.

- This sort of code is very common in Java.

```
public class Dog implements Comparable<Dog> {
    ...
    @Override
    public int compareTo(Dog uddaDog) {
        return size - uddaDog.size;
    }
}
```



CC BY NC SA

Dogs

Now that we've implemented `Comparable`, the code below works fine!

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new Dog("Grigometh", 200));
dogs.add(new Dog("Pelusa", 5));
dogs.add(new Dog("Clifford", 9000));
Dog maxDog = Collections.max(dogs);
```

CC BY NC SA

Polymorphism vs. Operator Overloading

The flavor of polymorphism we've just employed is sometimes called **subtype polymorphism**.

- A supertype (Comparable) specifies the capability (in this case, comparison).
- A **subtype overrides** the supertype's abstract method.
  - Java decides what to do at runtime based on the type of the object that is invoking the method.

This is very different than Python's polymorphism, where there is a universal > operator which can be overloaded.

CC BY NC SA

Compilation Error Puzzle

Lecture 11, CS61B, Fall 2026

Polymorphism vs. Function Passing

Comparable and Comparators

- Comparables
- **Compilation Error Puzzle**
- Comparators
- Comparables vs. Comparators

Summary

CC BY NC SA

Interfaces Quiz #1: hugcode.com/piano

```
public class DogLauncher {
  public static void main(String[] args) {
    ...
    Dog[] dogs = new Dog[]{d1, d2, d3};
  System.out.println(Collections.max(dogs));
  }
}
```

```
public class Dog
implements Comparable<Dog> {
  ...
  public int compareTo(Dog o) {
    return this.size
           - o.size;
  } ...
```

Q: If we omit `compareTo()`, which file will fail to **compile**?

A. DogLauncher.java
B. Dog.java
C. Collections.java
D. Comparable.java

```
public interface Collections {
    // the max function
    ...
    int cmp = items[i].
      compareTo(items[maxDex]);
    ...
  }...
}
```

CC BY NC SA

Interfaces Quiz #2: hugcode.com/tiger (Also attendance)

```
public class DogLauncher {
  public static void main(String[] args) {
    ...
    Dog[] dogs = new Dog[]{d1, d2, d3};
  System.out.println(Collections.max(dogs));
  }
}
```

```
public class Dog
implements Comparable<Dog> {
  ...
  public int compareTo(Dog o) {
    return this.size
           - o.size;
  } ...
```

Q: If we omit `implements Comparable<Dog>`, which file will fail to **compile**?

A. DogLauncher.java
B. Dog.java
C. Collections.java
D. Comparable.java

```
public interface Collections {
    // the max function
    ...
    int cmp = items[i].
      compareTo(items[maxDex]);
    ...
  }...
}
```

CC BY NC SA

Answers to Quiz

Problem 1: `Dog` will fail to compile because it does not implement all abstract methods required by `Comparable` interface. (And I suppose `DogLauncher` will fail as well since `Dog.class` doesn't exist)

Problem 2: `DogLauncher` will fail, because it tries to pass things that are not `Comparable`, and `Collections` expects `Comparables`.

- If we used `Override`, `Dog` will not compile, because we have an override tag but we're not overriding (if we omit `implements Comparable`).

CC BY NC SA

Comparators

Lecture 11, CS61B, Fall 2026

Polymorphism vs. Function Passing

Comparable and Comparators

- Comparables
- Compilation Error Puzzle
- **Comparators**
- Comparables vs. Comparators

Summary

CC BY NC SA

Natural Order

The term “Natural Order” is sometimes used to refer to the ordering implied by a `Comparable`’s `compareTo` method.

- Example: Dog objects (as we’ve defined them) have a natural order given by their size.



“Pelusa”, size: 5



“Grigometh”, size: 200



“Clifford”, size: 9000

CC BY NC SA

Alternate Sorting

May wish to order objects in a different way.

- Example: By Name.



“Clifford”, size: 9000



“Grigometh”, size: 200



“Pelusa”, size: 5

CC BY NC SA

Reminder: Key Functions in Python

In Python, we achieved the goal of specifying how to compare using **function passing**.

```
class Dog:
  def __init__(self, name, size):
    self.name = name
    self.size = size
  def __gt__(self, other):
    return self.size > other.size
```

```
def name_len(d):
  return len(d.name)

max(doglist, key=name_len)
```

By contrast, in Java, idiomatic code uses **subtype polymorphism**.

CC BY NC SA

The Comparator Interface

Java provides a `Comparator` interface for objects that are designed for comparing other objects.

```
public interface Comparator<T> {
    int compare(T o1, T o2);
    ...
}
```

Comparator<Dog>

Let's try building and using one for comparing Dogs based on name.

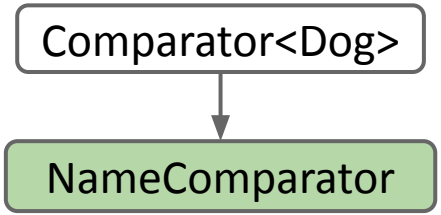
Note: The Comparator interface has a LOT of default methods. We won't talk about them.

CC BY NC SA

A Name Comparator

Java provides a `Comparator` interface for objects that are designed for comparing other objects.

```
public interface Comparator<T> {
    int compare(T o1, T o2);
    ...
}
```




```
public class NameComparator implements Comparator<Dog> {
    @Override
    public int compare(Dog a, Dog b) {
        return a.name.compareTo(b.name);
    }
}
```

CC BY NC SA

Comparator Quiz: hugcode.com/lemon

What will be the output of the code below?

IO.println(“Frank”.compareTo(“Zeke”)); // -negative number

```
Dog a = new Dog("Frank", 1);
Dog b = new Dog("Zeke", 1);
Comparator<Dog> nc = new Dog.NameComparator();
System.out.println(nc.compare(a, b));
```

```
public static class NameComparator implements Comparator<Dog> {
    @Override
    public int compare(Dog a, Dog b) {
        return a.name.compareTo(b.name);
    }
}
```

A. +1 B. Positive Number C. -1 D. Negative Number E. Zero

CC BY NC SA

Comparator Quiz

What will be the output of the code below?

```
Dog a = new Dog("Frank", 1);
Dog b = new Dog("Zeke", 1);
Comparator<Dog> nc = new Dog.NameComparator();
System.out.println(nc.compare(a, b));
```

```
public static class NameComparator implements Comparator<Dog> {
    @Override
    public int compare(Dog a, Dog b) {
        return a.name.compareTo(b.name);
    }
}
```

The result will be a **negative number**: “Frank” is alphabetically less than “Zeke”.

CC BY NC SA

Using a Comparator to Find the Maximum

We can use our `NameComparator` to find the dog with the maximum name.

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new Dog("Grigometh", 200));
dogs.add(new Dog("Pelusa", 5));
dogs.add(new Dog("Clifford", 9000));
Dog maxNameDog = Collections.max(dogs, new Dog.NameComparator());
```

Compare with Python below (using length of name rather than name itself).

```
def name_len(dog):
    return len(dog.name)
dogs = [Dog("Grigometh", 10),
        Dog("Pelusa", 5),
        Dog("Clifford", 9000)]
max(dogs, key=name_len)
```

This second argument is an object of type **`Comparator<Dog>`**

CC BY NC SA

Using a Comparator to Find the Maximum

We can use our `NameComparator` to find the dog with the maximum name.

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new Dog("Grigometh", 200));
dogs.add(new Dog("Pelusa", 5));
dogs.add(new Dog("Clifford", 9000));
Dog maxNameDog = Collections.max(dogs, new Dog.NameComparator());
```

In Java we package our comparison function inside of a Comparator object. We rely on **subtype polymorphism**.

Compare with Python below.

```
def name_len(dog):
    return len(dog.name)
dogs = [Dog("Grigometh", 10),
        Dog("Pelusa", 5),
        Dog("Clifford", 9000)]
max(dogs, key=name_len)
```

In Python we often **pass functions** directly.

CC BY NC SA

Awkward Instantiation

The NameComparator instantiation is awkward and aesthetically unpleasant (to me).

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new Dog("Grigometh", 200));
dogs.add(new Dog("Pelusa", 5));
dogs.add(new Dog("Clifford", 9000));
Dog maxNameDog = Collections.max(dogs, new Dog.NameComparator());
```

One fix is to add a static variable reference to a pre-instantiated NameComparator.

CC BY NC SA

Awkward Instantiation

The NameComparator instantiation is awkward and aesthetically unpleasant (to me).

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new Dog("Grigometh", 200));
dogs.add(new Dog("Pelusa", 5));
dogs.add(new Dog("Clifford", 9000));
Dog maxNameDog = Collections.max(dogs, new Dog.NameComparator());
```

One fix is to add a static variable reference to a pre-instantiated NameComparator. The usual naming convention is all caps:

```
public static NameComparator NAME_COMPARATOR = new NameComparator();
```

```
Dog maxNameDog = Collections.max(dogs, Dog.NAME_COMPARATOR);
```

CC BY NC SA

Bonus Slide: Lambdas

A more modern approach is to use a lambda to define the Comparator.

- The lambda below defines and instantiates a Comparator object that compares the name of two dogs.
- We will not teach Java lambdas in 61B, nor will you be expected to learn them.

```
List<Dog> dogs = new ArrayList<>();
dogs.add(new Dog("Grigometh", 200));
dogs.add(new Dog("Pelusa", 5));
dogs.add(new Dog("Clifford", 9000));

Comparator<Dog> dc = (d1, d2) -> d1.name.compareTo(d2.name);
Dog maxNameDog = Collections.max(dogs, dc);
```

CC BY NC SA

Comparables vs. Comparators

Lecture 11, CS61B, Fall 2026

Polymorphism vs. Function Passing

Comparable and Comparators

- Comparables
- Compilation Error Puzzle
- Comparators
- **Comparables vs. Comparators**

Summary

CC BY NC SA

Comparables vs. Comparators

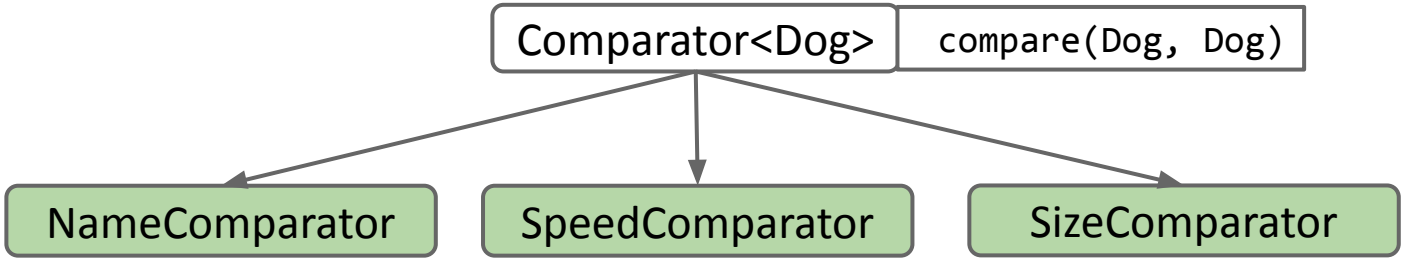
The Comparable interface specifies that a “natural order” exists.

- Instances of the class can compare themselves to other objects. Only one such order is possible.



The Comparator interface is used to compare extrinsically (by other classes).

- May have many such classes, each specifying one such order.



cc BY NC SA

Comparables vs. Comparators VS. Iterables vs. Iterators

Comparables and Comparators do not have the same relationship as Iterables and Iterators.

- `Comparable`: I can be compared to another object.
  - `int compareTo(T other)`
- `Comparator`: I can tell you how to compare two objects.
  - `int compare(T x1, T x2)`

- `Iterable`: I can give you an iterator.
  - `Iterator<T> iterator()`
- `Iterator`: I can feed you objects.
  - `boolean hasNext()`
  - `T next()`

CC BY NC SA

Summary

Lecture 11, CS61B, Fall 2026

Polymorphism vs. Function Passing

Comparable and Comparators

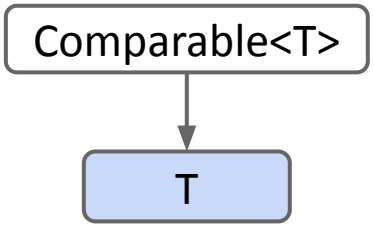
- Comparables
- Compilation Error Puzzle
- Comparators
- Comparables vs. Comparators

Summary

CC BY NC SA

Summary

Today we've seen **polymorphism** and **function passing**.



For comparing two objects using an intrinsic order:

- Python uses a form of polymorphism called **operator overloading**.
- Java instead uses **subtype polymorphism**.

Python is duck typed: Do not have to specify if > is available or not.

```
class Dog:
    def __init__(self, name, size):
        self.name = name
        self.size = size

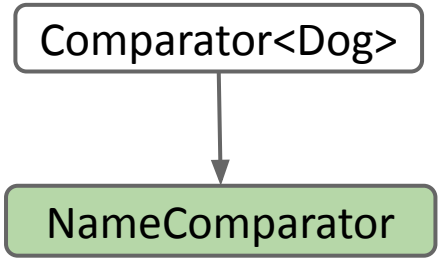
    def __gt__(self, other):
        return self.size > other.size
max(doglist)
```

```
public class Dog implements Comparable<Dog> {
    ...
    @Override
    public int compareTo(Dog uddaDog) {
        return size - uddaDog.size;
    }
}
Collections.max(dogs);
```

CC BY NC SA

Summary

Today we've seen **polymorphism** and **function passing**.



For comparing two objects using an alternate order:

- Python uses **function passing**. You provide a key function.
- Java instead (usually) uses **subtype polymorphism**, just like for intrinsic orders. We package the comparison function in a "Comparator" object.

```
class Dog:
    def __init__(self, name, size):
        self.name = name
        self.size = size

def name_len(dog):
    return len(dog.name)

max(doglist, key=name_len)
```

```
public static class NameComparator
          implements Comparator<Dog> {
    @Override
    public int compare(Dog a, Dog b) {
        return a.name.compareTo(b.name);
    }
}
Collections.max(dogs, new NameComparator());
```

CC BY NC SA